Full-Stack User Management CRUD System

Objective

Build a **complete user management system** with a **Next.js frontend** and a **Node.js + Express + MongoDB backend**.
The system should allow creating, editing, updating passwords, deleting, and listing users with pagination, search filters, and role-based selection.

The goal of this task is to evaluate your understanding of **full-stack architecture**, **frontend integration with backend APIs**, and **clean code structure**.

Tech Stack

Frontend:

- Next.js (latest version)
- React Hooks and Context or Redux (optional)
- Axios (for API requests)
- TailwindCSS or any modern CSS framework
- Pagination and filtering on UI level

Backend:

- Node.js
- Express.js
- MongoDB
- Mongoose (ORM)
- JWT or basic middleware auth (optional but preferred)
- Proper project structuring with controllers, routes, and models

---

Functional Requirements

1. User Management (CRUD)

Create a **user management table** on the frontend that allows:

- **Create User**
    - Fields: `name`, `email`, `role`, `password`, `confirm password`
    - Role must be selected from a **dropdown** with 5 hardcoded options (e.g., `Admin`, `Manager`, `Director`, `Supervisor`, `Operator`).
    - Validate password and confirm password match before submitting.
- **Read (List Users)**
    - Display all users in a responsive table.
    - Each row should include:
      `Name`, `Email`, `Role`, `Created Date`, and **Actions** (`Edit`, `Delete`, `Update Password`).
    - Support **pagination** with `page` and `limit` query parameters.
    - Default: 10 items per page, with options to change items per page.
- **Update User**
    - Allow editing user details (`name`, `email`, `role`).
    - Update operation should reflect immediately on the table after successful API call.
- **Update Password**
    - Separate action for updating the password of a user.
    - Fields: `New Password`, `Confirm Password`

    - Ensure password confirmation validation on both frontend and backend.

- **Delete User**
    - Implement delete functionality with confirmation popup or modal.

---

2. Search and Filter Features

The table should include **search and filter** capabilities:

- **Search by Name:**
  A search input to filter users by name (case-insensitive).

- **Filter by Role:**
  A dropdown to select and filter users by role.

- **Filter by Date Range:**
  Use a date picker or two input fields (`fromDate` and `toDate`) to filter users by creation date range.

The frontend should send these filters as **query parameters** to the backend API.

Example request:

```
GET
/api/users?search=John&role=Manager&from=2025-10-01&to=2025-10-09&page=2&limit=10
```

---

3. Backend Requirements

Folder Structure (Mandatory)

Use a clean and scalable project structure like:

```
/backend
  /config
    db.js
  /controllers
```

```
    userController.js
  /models
    userModel.js
  /routes
    userRoutes.js
  /middlewares
    errorHandler.js
  /utils
    helpers.js
server.js
```

Model (Mongoose Schema Example)

User schema fields:

- `name` (String, required)
- `email` (String, unique, required)
- `password` (String, required)
- `role` (String, required)
- `createdAt` (Date, default: Date.now)

Controllers

Create controller functions for:

- `createUser`
- `getUsers` (with search, filters, pagination)
- `updateUser`
- `updatePassword`
- `deleteUser`

Ensure proper error handling and validation in each controller.

Routes

```
POST    /api/users          → Create new user
GET     /api/users          → Get all users (with filters + pagination)
PUT     /api/users/:id      → Update user details
PATCH   /api/users/:id/password  → Update password
DELETE  /api/users/:id      → Delete user
```

Database Connection

- Use Mongoose for MongoDB connection.
- Keep DB URI and credentials in `.env` file.
- Handle database connection and errors in `/config/db.js`.

---

4. Frontend Requirements

Pages & Components

- **UserTable.js** – Displays all users with pagination and filters.
- **UserForm.js** – For adding or editing users.
- **UpdatePasswordModal.js** – For password update UI.
- **Filters.js** – For search, role dropdown, and date range filter.
- **Pagination.js** – For page navigation and item per page control.

Functionalities

- Use **Axios** to connect with the backend API.
- Handle all CRUD operations with API integration.
- Show proper **loading** and **error** states.

- Ensure responsive design and clean UI.
- Implement pagination controls at the bottom of the table.

---

5. Pagination Logic Example

Frontend must handle:

- Page navigation (Previous / Next)
- Item per page selection (5, 10, 20, etc.)

Backend must return paginated response like:

```
{
  "page": 2,
  "limit": 10,
  "totalPages": 5,
  "totalItems": 45,
  "data": [...]
}
```

---

6. Validation

- Backend should validate required fields and send meaningful error messages.
- Frontend should show validation errors before API call if fields are empty or invalid.
- Passwords must be validated to match before submission.

---

7. Deliverables

- Fully functional **Next.js frontend** with all CRUD and filters implemented.

- Fully structured **Node.js + Express backend** with MongoDB connection.
- README file explaining:
    - How to set up and run both frontend and backend.
    - API endpoints overview.
    - Example screenshots of UI.
- Optional: Use environment variables for API URLs.

---

8. Evaluation Criteria

Area	Description
Code Structure	Proper separation of concerns and clean folder organization
Functionality	All CRUD and filter operations working correctly
Pagination	Working pagination on both backend and frontend
Validation	Proper client-side and server-side validation
UI/UX	Clean, professional table layout and responsive design
Performance	Efficient querying and API response handling
Documentation	Clear README and well-commented code